

# Extension roadmap

Grouped by what unlocks what. Every deferred item has a trigger condition and estimated cost. Full deferred-work catalogue lives in `architecture/FUTURE_WORK` §2.

## 1. Immediate follow-ups (finish-line work)

Small items that don't require new infrastructure. Cheap to ship, close visible gaps.

### 1.1 Full-results paginated view + CSV download

**Effort:** ≈2–3 hours. **What:** The overflow response already carries the row count + sample; the AnswerCard already shows disabled buttons for "View full results" and "Download CSV". Wire the endpoints:

- `GET /results/{turn_id}?page=N&size=M` — re-executes the stored SQL with `OFFSET / LIMIT`
- `GET /results/{turn_id}/csv` — streamed CSV via SQLAlchemy row-by-row (no in-memory materialisation)
- Frontend route `/results/:turnId` with paginated table
- `chat.py` gets a `get_turn(turn_id)` helper for the Mongo lookup

**Value:** completes the D-019 progressive-disclosure story to a full user-facing UX. Every result size class fully navigable without needing SQL knowledge.

### 1.2 History fetch on page load

**Effort:** ≈1.5 hours. **What:** Every turn is already persisted to MongoDB. Add a `GET /history` endpoint (with optional `session_id` filter for future auth) + `useEffect` on frontend mount that hydrates the `Turn[]` state from it. **Value:** browser refresh restores the visual conversation instead of losing it. Server-side audit trail already exists; this closes the UX loop.

### 1.3 Retry-loop controlled test in the eval harness

**Effort:** ≈30 minutes. **What:** The retry code is verified by inspection but hard to trigger naturally with `gpt-4o` at `temp=0` + the schema disambiguation prompt. Day 6 eval harness would construct a deterministic failure scenario (mock Execute node returning a specific error string) to exercise the retry state machine end-to-end. **Value:** closes a Day 4 promissory note; makes the retry guarantee reproducibly verified rather than reasoned about.

## 2. Feature growth

---

New user-visible capabilities. Real product roadmap items.

### 2.1 Multi-database and warehouse support

**What:** Snowflake, BigQuery, Redshift support. Per-source schema retrieval and per-source `agent_readonly` role. `business_db.py` gets a source router; RAG few-shots get per-source scoping. **Value:** unlocks any company that isn't Postgres-only (i.e. most enterprises with a warehouse).

### 2.2 Auto-charting

**What:** infer when a result is better shown as a bar / line / pie chart from column types and cardinality; render inline with the answer instead of just a table. **Value:** analyst gets an insight faster; the tool feels less like a SQL wrapper and more like a lightweight BI copilot.

### 2.3 Clarification loop

**What:** when a question is genuinely ambiguous (multiple valid interpretations — "best customer" by what metric?), ask a clarifying question back instead of picking one silently or refusing outright.

**Current state:** ambiguous questions are marked as `either_ok_or_refuse` in the eval — either behaviour is acceptable. Clarification would make this a third, better option. **Value:** turns "sometimes wrong, sometimes refused" into "always right after one clarifying exchange."

### 2.4 Feedback loop

**What:** thumbs up / down on answers; positive feedback appends the question + SQL as a new few-shot into `examples.json` (or, at scale, into a review queue). The system learns your team's conventions over time. **Value:** the RAG layer becomes self-improving. Every domain-specific answer a user marks correct becomes a training example for future similar questions.

### 2.5 Human-in-the-loop approval

**What:** for higher-risk query classes (e.g. queries touching sensitive columns if such are ever added), require a second human OK before execution. **Value:** enterprise / regulated-industry buyers require this. Not needed for the current demo scope.

## 3. Scale and production growth

---

Items that become relevant as traffic grows or the product goes external. Each has an explicit trigger condition — none of them should be built preemptively.

### 3.1 Multi-user auth + per-user chat scoping

**Trigger:** first external rollout beyond one team, OR any external exposure. **Effort:**  $\approx 1-2$  days. **What:** JWT-based auth at FastAPI layer; `user_id` threaded through Mongo `turns` documents; per-user history views. The isolation model already supports this — the MongoDB turn document just needs a `user_id` field, and the `/history` endpoint (§1.2) already accepts a `session_id` query param that can become `user_id`.

### 3.2 Redis semantic cache in front of the LLM

**Trigger:** LLM cost  $> \$50/\text{mo}$  OR observed repeat-question rate  $> 30\%$ . **Effort:**  $\approx 1$  day. **What:** hash + embed each incoming question; look up in Redis for a "close enough" prior answer; return the cached answer without invoking the graph. On realistic analytics traffic, kills 30–70% of LLM calls. **Impact when needed:** 30–70% reduction in LLM calls; latency drops from  $\approx 2\text{s}$  to  $\approx 50\text{ms}$  on cache hits.

### 3.3 PgBouncer for Postgres connection pooling

**Trigger:** concurrent Postgres connections consistently  $> 50$ . **Effort:**  $\approx 4$  hours. **What:** connection pooler in transaction-pooling mode; the app opens cheap logical connections, PgBouncer maintains few physical ones to Postgres. At demo scale (SQLAlchemy `pool_size=5+5`) not needed; at real scale essential.

### 3.4 Read replicas for Postgres

**Trigger:** primary CPU sustained  $> 70\%$ , OR read latency degrades under load. **Effort:**  $\approx 1$  day. **What:** primary handles writes (RAG ingest, admin); replicas handle the agent's `SELECT`s. Trivially safe because `agent_readonly` role is read-only — no possibility of a rogue write to a replica. **Impact:** near-linear read scaling.

### 3.5 LLM provider failover with circuit breaker

**Trigger:** first user-visible outage caused by OpenAI, OR SLA target  $> 99.5\%$ . **Effort:**  $\approx 1$  day. **What:** primary OpenAI GPT-4o with secondary Anthropic Claude Haiku fallback; automatic switch on 3 consecutive 429s or 5xx. **Impact:** removes single-provider risk; degraded-quality answers during primary outage rather than hard failures.

### 3.6 Rate limiting at the load balancer

**Trigger:** first exposure to external users, OR observed pattern of one user swamping the system. **Effort:**  $\approx 2$  hours. **What:** per-API-key / per-user request rate limits at the ingress layer (NGINX / ALB rules). **Impact:** prevents one heavy user from starving others; caps LLM cost per user.

### 3.7 CDN for the frontend

**Trigger:** first production deploy. **Effort:**  $\approx 2$  hours. **What:** static frontend build ( `vite build` ) served via CloudFront / Fastly instead of Vite dev server. The current dev-server model is not a production server. **Impact:** removes single-server bottleneck for static assets; enables global edge caching.

### 3.8 Live schema introspection with curated allowlist + business-context overlay

**Trigger:** business database gets an independent release cadence from the AI app, OR schema grows past  $\approx 10$  tables. **Effort:**  $\approx 100$  lines of code plus a startup validation step. **What:** replace the hardcoded schema-in-prompt with three layers:

1. Introspect `information_schema` at boot to get the live schema from the DB
2. Apply a code-reviewed allowlist that filters which tables/columns the LLM is permitted to see (excludes internal audit columns, PII flags, cost/margin fields, deprecated columns)
3. Overlay a curated business-context dictionary that attaches human notes to specific columns

**Impact:** schema changes in the business app propagate automatically; sensitive additions default to hidden until reviewed; dropped columns fail loudly at boot so drift is caught immediately.

### 3.9 Self-hosted Langfuse

**Trigger:** trace volume exceeds Langfuse Cloud free tier, OR data-residency requirements. **Effort:**  $\approx 2$  hours. **What:** run Langfuse as a compose service instead of using Langfuse Cloud. **Impact:** no vendor lock-in on observability; local trace storage.

## 4. Framework and architecture evolution

---

Bigger structural changes. Only warranted when the underlying assumption changes.

### 4.1 Google ADK port with measured comparison

**Trigger:** client wants measured LangGraph-vs-ADK numbers, OR product pivots toward Gemini/Vertex ecosystem, OR multi-agent orchestration becomes a first-class feature. **Effort:**  $\approx 2-3$  hours if it goes smoothly. **What:** port the 5-node pipeline to Google ADK behind the `FRAMEWORK=adk` env var switch that's already in `config.py`. Reuse the existing guardrails, DB layer, RAG retrieval, Langfuse instrumentation unchanged — only the orchestration differs. **Impact:** turns `O5_FRAMEWORK_COMPARISON` from qualitative to measured. Provides real numbers for a framework decision the buyer might want to make themselves.

## 4.2 RBAC (role-based access control)

**Trigger:** any regulated-industry deployment (healthcare, finance) or multi-tenant with data segregation requirements. **Effort:**  $\approx 1$  week. **What:** different users see different tables / rows / columns. Backed by per-user read-only Postgres roles, per-tenant RAG scoping, and query-time filter injection based on user context. **Impact:** unlocks vertical SaaS in regulated industries where row-level security is non-negotiable.

## 4.3 OpenTelemetry beyond Langfuse

**Trigger:** need to debug latency in non-LLM paths, OR broader observability platform arrives (Jaeger / Tempo). **Effort:**  $\approx 1$  day. **What:** OpenTelemetry SDK  $\rightarrow$  Tempo/Jaeger for non-LLM spans (DB queries, HTTP calls, internal function boundaries). **Impact:** full-stack per-request timing visibility, correlate LLM traces with DB/HTTP work.

## 5. What we deliberately chose NOT to build (and why)

---

Not deferred with a trigger — declined outright. Some things are wrong for the shape of this problem regardless of scale.

- **Pydantic for LangGraph state** — TypedDict is idiomatic in LangGraph, zero runtime overhead. Pydantic's per-transition validation cost grows with state size. See design decision D-005.
- **langchain-core chat abstractions** — thin `llm.py` wrapper gives provider swap + testability without the LangChain surface area. See D-006.
- **prompts/\*.txt files** — inline constants keep the prompt visible with the code that uses it. See D-014.
- **Kubernetes for the current build** — Docker Compose fits the local-demo scope. K8s adds ops cost with no benefit at this scale.
- **Custom fine-tuned SQL model** — RAG-assisted prompting is cheaper, faster to iterate, and good enough at this scope. Fine-tune only when frontier-model cost is prohibitive AND domain SQL patterns are stable enough to train on.
- **WebSocket transport** — SSE fits the server $\rightarrow$ client flow exactly. Revisit when cancel-mid-query becomes a requirement. See D-012.
- **Multiple backend services** (agent-api, ingest-worker, admin-api) — single FastAPI monolith fits the current scope; splitting multiplies ops surface without payoff. Revisit when profiling shows one path starving another.
- **Typewriter / artificial pacing on the Explain stream** — contradicts the point of SSE streaming; real tokens are already arriving as fast as the network allows. The blinking emerald caret at end-of-text is the streaming affordance instead.

## 6. Priority sequencing recommendation

---

If you're picking what to build next after receiving this delivery:

1. **§1.1 — Full-results paginated view + CSV download** (≈3h). Closes the disabled-buttons UX gap that reviewers will notice on any 500-row query.
2. **§1.2 — History fetch on page load** (≈1.5h). Refresh no longer loses the conversation. Small but high-visibility UX win.
3. **§3.1 — Multi-user auth** (≈1–2 days). The bridge from single-user demo to real multi-tenant deployment. Everything after this depends on it.
4. **§4.1 — Google ADK port** (≈3h) OR **§2.4 — Feedback loop** (≈1–2 days). Framework flexibility signal vs. self-improving RAG layer. Pick based on which story matters more for the next milestone.

The scale/production items (§3.2 onward) only become relevant when their trigger conditions fire. Building them preemptively is the #1 way small teams burn runway.